

PYTHON API INTEGRATION & JSON HANDLING REPORT

Name :

DEETCHANA R

Candidate ID :

BS/REG/119987

Domain :

Python Developer Intern

1. Objective

To understand how Python communicates with external APIs and handles JSON data by fetching user information from a public API, parsing JSON responses, applying search/filter logic, and handling API-related errors.

2. Requirements Covered

- Fetch data using Requests library
- Parse JSON responses
- Apply filtering/search logic
- Handle API errors using try-except
- Display filtered results
- Automate file operations (rename, move, delete)

3. Development Process (Step by Step)

Step 1 - Import Requests Library

```
import requests
```

Step 2 - Fetch Data from API

```
url = "https://jsonplaceholder.typicode.com/users"
```

```
response = requests.get(url)
```

Step 3 - Check Request Status

```
response.raise_for_status()
```

Step 4 - Parse JSON Response

```
users = response.json()
```

Step 5 - Apply Search Logic

```
search_city = "South Christy"
```

Step 6 - Display Matching Records

for user in users:

```
    if user["address"]["city"] == search_city:
        print("Name:", user["name"])
        print("Email:", user["email"])
        print("City:", user["address"]["city"])
```

Step 7 - Handle API Errors

except requests.exceptions.RequestException as e:

```
    print("API Error:", e)
```

4. Final Python Code

```
import requests
```

```
try:
```

```
    # API URL
```

```
    url = "https://jsonplaceholder.typicode.com/users"
```

```
    # Send GET request to API
```

```
    response = requests.get(url)
```

```
    # Check if request was successful
```

```
    response.raise_for_status()
```

```
    # Convert JSON response into Python data
```

```
    users = response.json()
```

```
print("User List:\n")

# Search users from a specific city
search_city = "South Christy"

# Display matching user information
for user in users:

    if user["address"]["city"] == search_city:

        print("Name:", user["name"])
        print("Email:", user["email"])
        print("City:", user["address"]["city"])
        print("-" * 30)

# Handle API-related errors
except requests.exceptions.RequestException as e:

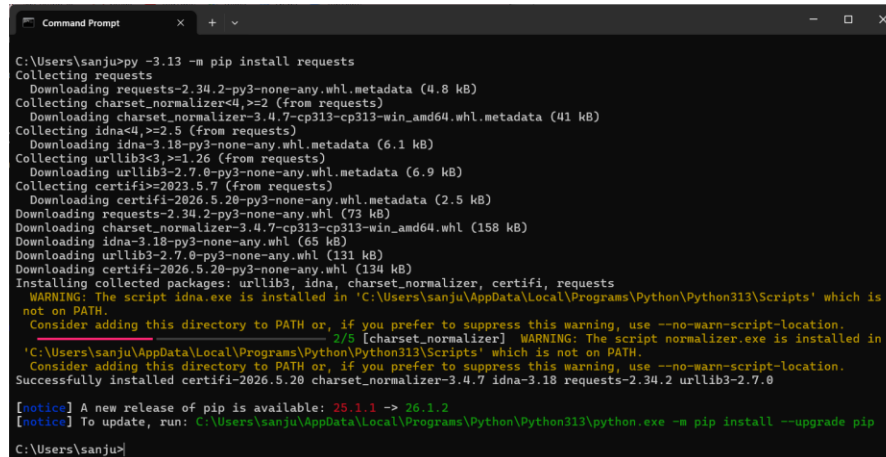
    print("API Error:", e)

# Handle other unexpected errors
except Exception as e:

    print("An error occurred:", e)
```

5. Screenshots with Captions

Figure 1 : Installing Requests Library

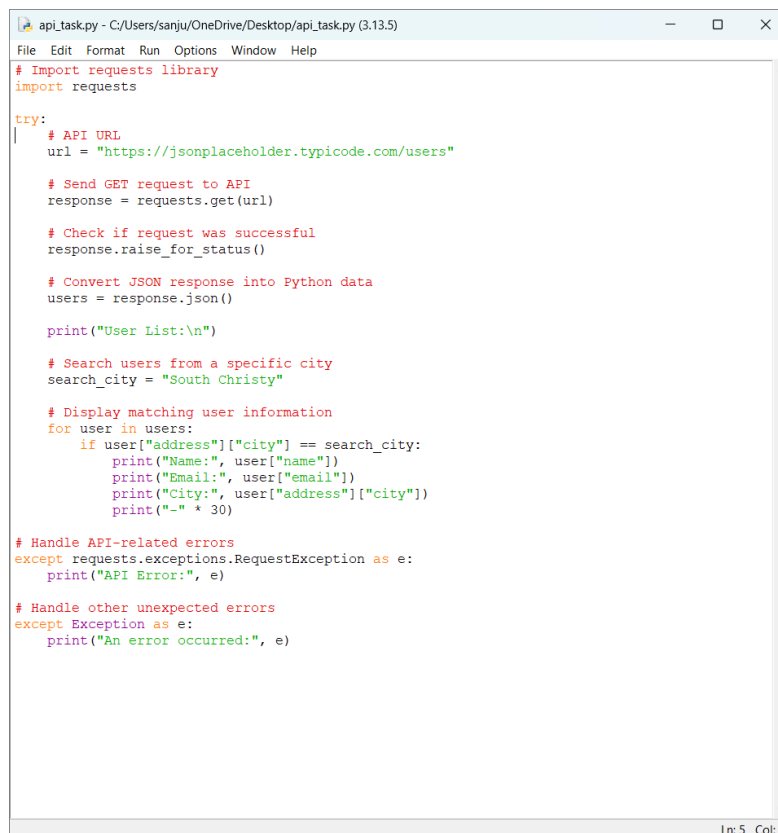


```
C:\Users\sanju>py -3.13 -m pip install requests
Collecting requests
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting charset-normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.4.7-cp313-cp313-win_amd64.whl.metadata (41 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.18-py3-none-any.whl.metadata (6.1 kB)
Collecting urllib3<3,>=1.26 (from requests)
  Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2023.5.7 (from requests)
  Downloading certifi-2026.5.20-py3-none-any.whl.metadata (2.5 kB)
Downloading requests-2.34.2-py3-none-any.whl (72 kB)
Downloading charset_normalizer-3.4.7-cp313-cp313-win_amd64.whl (158 kB)
Downloading idna-3.18-py3-none-any.whl (65 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Downloading certifi-2026.5.20-py3-none-any.whl (134 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
WARNING: The script idna.exe is installed in 'C:\Users\sanju\AppData\Local\Programs\Python\Python313\Scripts' which is
not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
2/5 [charset_normalizer] WARNING: The script normalizer.exe is installed in
'C:\Users\sanju\AppData\Local\Programs\Python\Python313\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed certifi-2026.5.20 charset_normalizer-3.4.7 idna-3.18 requests-2.34.2 urllib3-2.7.0

[notice] A new release of pip is available: 25.1.1 -> 26.1.2
[notice] To update, run: C:\Users\sanju\AppData\Local\Programs\Python\Python313\python.exe -m pip install --upgrade pip

C:\Users\sanju>
```

Figure 2 : Python Code for API Integration and JSON Handling



```
api_task.py - C:\Users\sanju\OneDrive\Desktop\api_task.py (3.13.5)
File Edit Format Run Options Window Help

# Import requests library
import requests

try:
    # API URL
    url = "https://jsonplaceholder.typicode.com/users"

    # Send GET request to API
    response = requests.get(url)

    # Check if request was successful
    response.raise_for_status()

    # Convert JSON response into Python data
    users = response.json()

    print("User List:\n")

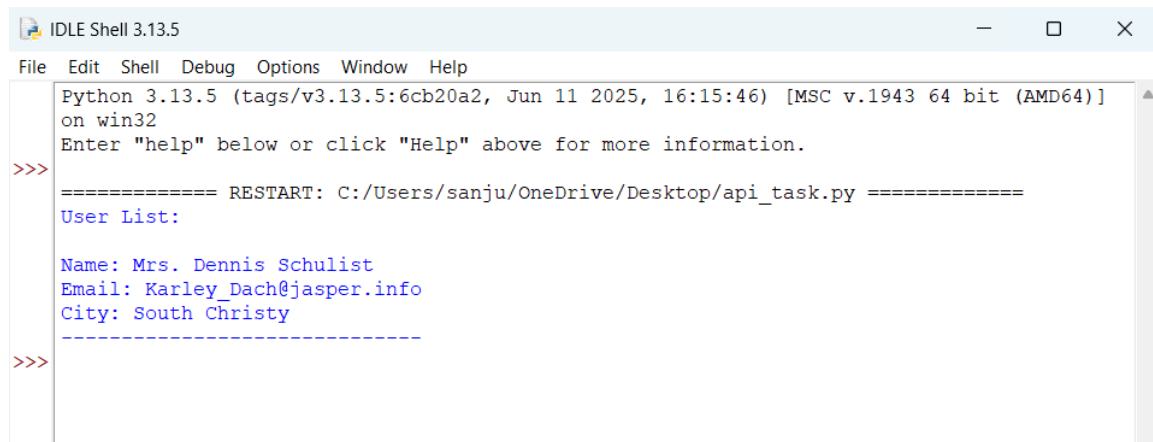
    # Search users from a specific city
    search_city = "South Christy"

    # Display matching user information
    for user in users:
        if user["address"]["city"] == search_city:
            print("Name:", user["name"])
            print("Email:", user["email"])
            print("City:", user["address"]["city"])
            print("-" * 30)

# Handle API-related errors
except requests.exceptions.RequestException as e:
    print("API Error:", e)

# Handle other unexpected errors
except Exception as e:
    print("An error occurred:", e)
```

Figure 3 : API Program Output



```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sanju/OneDrive/Desktop/api_task.py =====
User List:
Name: Mrs. Dennis Schulist
Email: Karley_Dach@jasper.info
City: South Christy
-----
>>>
```

6. Conclusion

This project successfully demonstrated API Integration and JSON Handling using Python. The Requests library was used to communicate with an external API, JSON data was parsed into Python objects, search and filtering logic was applied, and API errors were handled using exception handling techniques. All project requirements were completed successfully.